定时器 / 定时器按键控LED流水灯模式 & 定时器时钟

对应江协科技《51单片机入门教程-2020版》BV1Mb411e7re 第7章 · 范围 = P17 [7-1]定时器、P18 [7-2]定时器按键控制LED流水灯模式&定时器时钟 · 本章目标：理解定时器是单片机内部资源、靠系统时钟计数而非空转延时；弄清"时钟→计数单元→中断系统"工作流程与模式1(16位)；会算 1ms 定时初值(65536−1000=0xFC18)并配置 TMOD/TCON/TH0/TL0/IE；会写 Timer0_Init 与 interrupt 1 中断服务函数、每次手动重装初值；用定时器做流水灯节拍(按键切模式)与 LCD1602 时钟 · 不覆盖：T1/T2、模式0/2/3、外部计数(C/T=1)、中断优先级嵌套深入等后续内容

本节范围

前几章的"等"都是 CPU 空转 `Delay()` ——等着的时候什么也干不了（流水灯转起来就扫不到按键）。定时器换个思路：它是**单片机内部的一个小闹钟**，靠系统时钟每 1μs 自动给计数器 +1，加到溢出就主动敲门（中断）叫 CPU 来干活，CPU 在没敲门时可以照常扫按键、刷显示。主线只有一条——**先配好 TMOD 选模式1(16位)、装进定时初值、开 ET0/EA 中断、TR0=1 启动；之后每到 1ms 硬件自动触发 interrupt 1，进去手动重装初值并数满 1000 次就是 1 秒**。

> **学习提示：**本章最容易错的是"初值怎么算"和"忘了重装"。记住口径：**12MHz、12T 分频 → 计数脉冲 1μs；模式1 一次最多 65536μs≈65.5ms，定 1 秒要每 1ms 中断、在中断里数 1000 次**。1ms 初值 = 65536−1000 = 64536 = **0xFC18**（TH0=0xFC、TL0=0x18）。模式1 不是自动重装，**每次进中断第一句话就要重新装 TH0/TL0**，否则第二次起变成 65ms。手算 0xFC17 与计算器 0xFC18 差 1μs，以后直接用 STC-ISP 定时器计算器生成、不硬抄。



实物图 · 普中 HC6800-ES V2.0 开发板整板（课程同款）：定时器是芯片内部资源，无需外接电路，复用板上 12MHz 晶振 · 图源：网络搜索

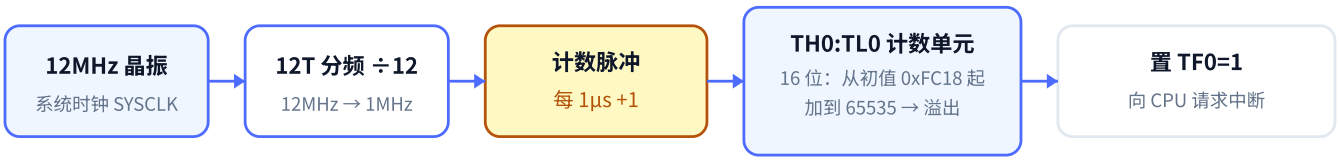
核心知识点

内容	条件 / 说明	易错

定时器定位	单片机**内部电路**（把芯片抠下来单独通电就能用），区别于按键/数码管/LCD 等 IO 口控制的外部设备；两大作用：计时系统、替代长 delay 让 CPU 不空等（P17 02:09–05:00）	用定时器后 CPU 可边流水边扫按键
定时器个数	STC89C52 有 **T0/T1/T2** 三个；T0/T1 兼容传统 8051，T2 是本型号新增；不同型号个数不同，换芯片先查手册（P17 06:55、09:30）	别默认所有芯片都有 T1
工作流程	像内部小闹钟：时钟每来一个脉冲计数单元 +1；加到溢出(最大值)时向中断系统发申请，程序跳进中断服务函数执行定时任务（P17 11:14–13:41）	—
4 种模式	模式0=13位、**模式1=16位(最常用，本章只讲它)**、模式2=8位自动重装、模式3=双8位（P17 15:39）	本章 M1M0=01
计数单元	**TH0(高字节)+TL0(低字节)拼 16 位**，最大计 65535；每个时钟脉冲 +1，加到 65535 后下一脉冲**溢出**，硬件置 **TF0=1** 请求中断（P17 18:48–20:37）	溢出后硬件自动置 TF0，CPU 响应后自动清 0
时钟来源/分频	本板晶振 **12MHz**；51 默认 **12T(12分频)**：12MHz÷12=1MHz，即计数脉冲周期=**1μs**，每 1μs 计数 +1（P17 24:31–25:59）	C/T=0 用系统时钟(定时)；=1 接外部引脚(计数)，本章 C/T=0
最大定时	16 位一次溢出最长 **65536μs≈65.5ms**，定不了 1 秒；办法是**每次定 1ms 中断、中断里数 1000 次**再做动作（P18 06:37）	单次定不了秒级，要软件累计
1ms 初值	计数器要从初值数 1000 个就溢出 → **初值 = 65536−1000 = 64536 = 0xFC18**；TH0=64536/256=252=**0xFC**，TL0=64536%256=24=**0x18**（P18 08:51、09:33、28:30）	手算 65535−1000=64535 得 0xFC17，比计算器差 1μs，以后用 STC-ISP 计算器
TMOD	工作模式寄存器，**不可位寻址、必须 8 位整体赋值**；高 4 位管 T1、低 4 位管 T0；本章 GATE=0、C/T=0、M1M0=01(模式1)（P17 51:57–53:08）	直接 TMOD=0x01 会清掉 T1 的高 4 位
与或式赋值	T0/T1 并存时只改低 4 位：`TMOD&=0xF0;`(高4位不变低4位清0)、`TMOD|=0x01;`(最低位置1)（P18 19:12–23:40）	这是保留 T1 配置的标准写法
TCON	**TR0=1 启动计数、TR0=0 停**；**TF0**=T0 溢出标志，硬件自动置 1/清 0，软件只读不写（查询法读它）（P17 49:11–50:16）	启动前先 TF0=0 防误触发

IE 中断使能	**EA=1 开总中断**(EA=0 全堵)、**ET0=1 开 T0 溢出中断**；PT0=0 默认低优先级（P17 58:16–01:01:08、P18 11:38）	只写 ET0 不写 EA 也进不来
中断服务函数	`void Timer0_Routine() interrupt 1`；T0 的**中断号=1**；interrupt 是关键字（P18 12:59）	主函数没调用它，溢出时硬件自动跳进来
手动重装	模式1 **不是自动重装**：溢出后从 0 开始数，下次就变成 65ms；所以**每次进中断都要手动重装 TH0/TL0**（像把沙漏翻回来）（P18 17:30、32:51）	忘了重装 → 第二次起定时全错
查询 vs 中断	查询法=主循环不断读 TF0 是否变 1；中断法=溢出自动跳 ISR；一般用中断法（P17 40:54）	本章综合例都用中断法
STC-ISP 计算器	参数：系统频率 **12MHz、定时长度 1ms、定时器 0、模式 16 位、时钟 12T**；生成代码第一句配 AUXR，**89C51 没这寄存器要删掉**，再手动补 ET0=1、EA=1（P18 25:30–32:15）	最小化重开计算器参数会复位，选前复查 12T/16位/1ms
循环移位	`<intrins.h>`：`_crol_(val,n)` 循环左移 n 位、`_cror_(val,n)` 循环右移；移出去的位补到另一端（P18 47:30）	流水灯用它让亮位循环跑动
综合例 1 逻辑	main 扫按键 `Key()`：K1 按下 LED_Mode 在 0/1 间切；ISR 每 500ms `if(LED_Mode==0) P2=_crol_(P2,1); else P2=_cror_(P2,1);`；P2 初值 0xFE（P18 46:29–52:17）	按键引脚是 P3(K1=P3.1)，P2 是 LED
综合例 2 时钟	ISR 里只做短任务：`Sec++; if(Sec>=60){Sec=0;Min++;} if(Min>=60){Min=0;Hour++;} if(Hour>=24){Hour=0;}`；LCD 刷屏这种慢操作放 while(1) 主循环（P18 59:34、58:23）	显示布局：时列1-2、冒号列3、分列4-5、冒号列6、秒列7-8

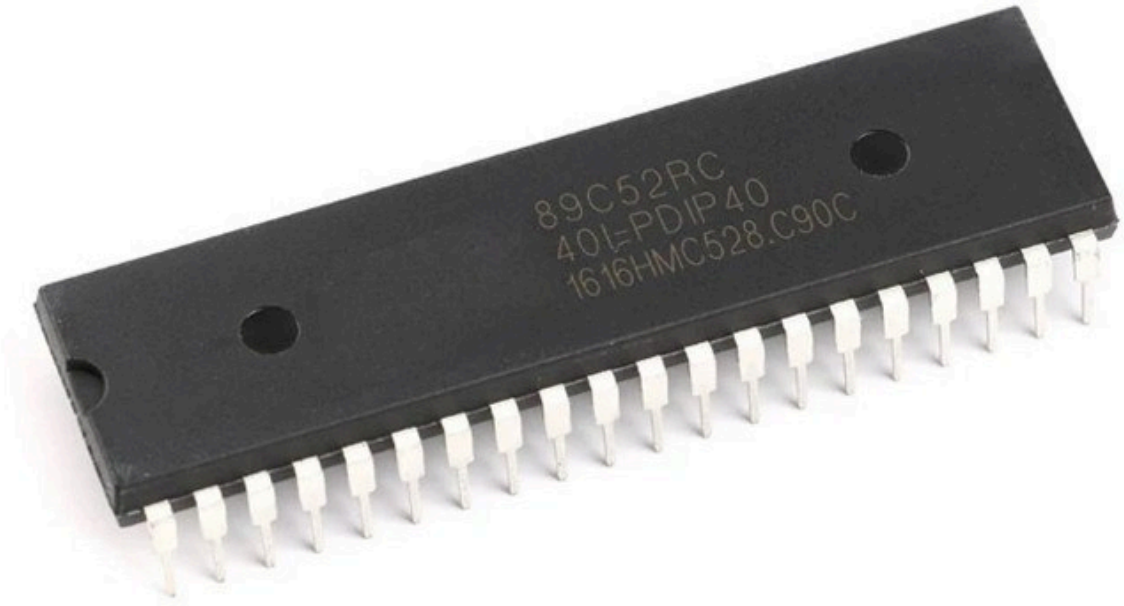
一句话记牢：12MHz + 12T = 每 1μs 计数 +1；模式1 从初值数到 65535 溢出触发 interrupt 1。1ms 初值 65536−1000=**0xFC18**（TH0=0xFC、TL0=0x18）；配好 TMOD(与或式)/TH0/TL0/ET0/EA 后 TR0=1 启动，**进中断第一件事就是重装 TH0/TL0**，否则第二次起定时变 65ms。



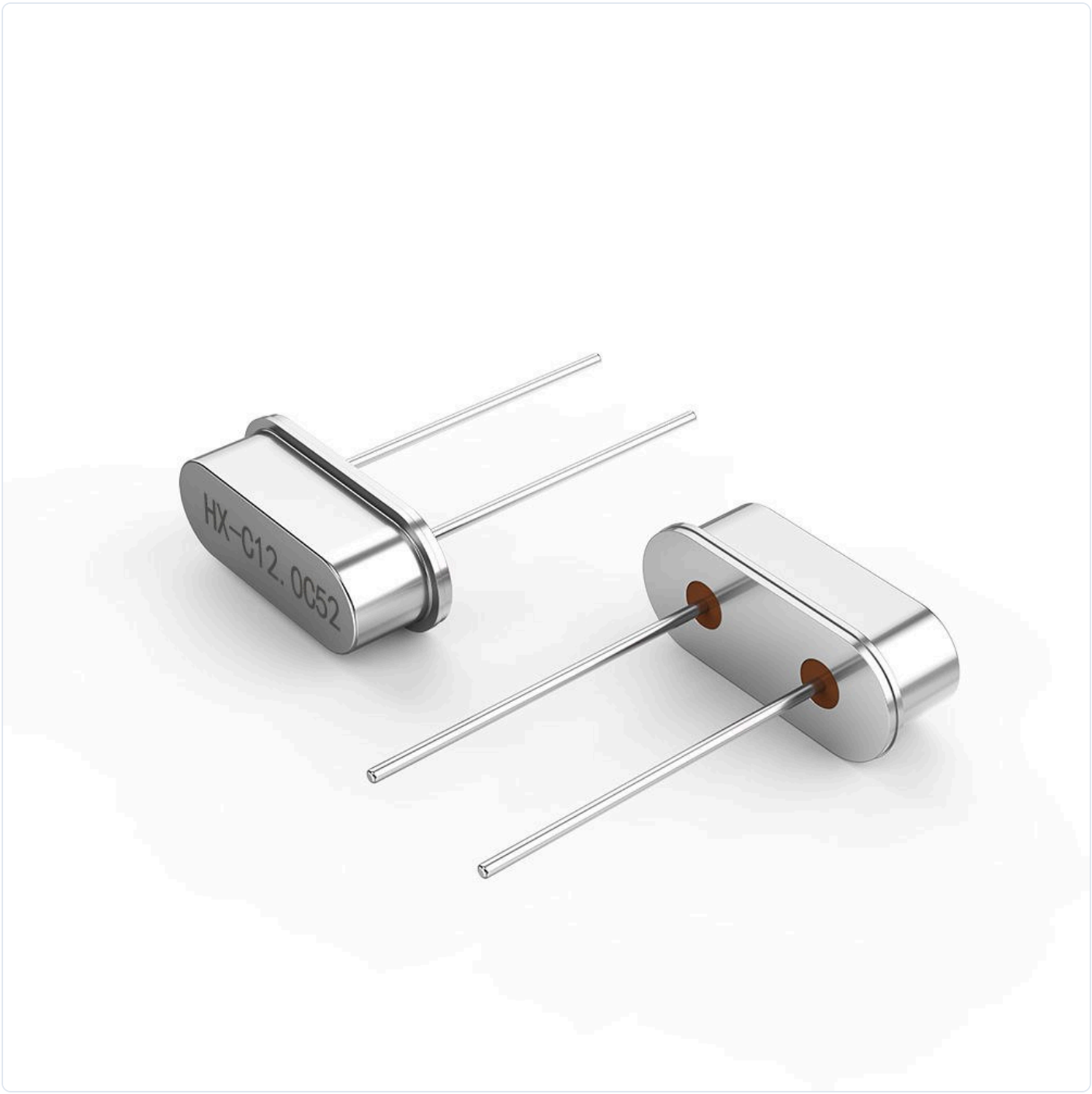
溢出 → TF0=1 → 硬件自动跳进 interrupt 1 中断服务函数

这就是"小闹钟"：不用 CPU 空转，闹钟响了才来处理（P17 24:31、P18 06:37）

图 ①　定时器系统框图：12MHz 晶振经 12T 分频得 1MHz(1μs) 计数脉冲，TH0:TL0 16 位计数器每脉冲 +1，溢出置 TF0=1 请求中断（P17 24:31–25:59、P18 06:37）。



实物图 · STC89C52 主控芯片：内部集成 T0/T1/T2 三个定时器，定时器是它的内部资源 · 图源：网络搜索

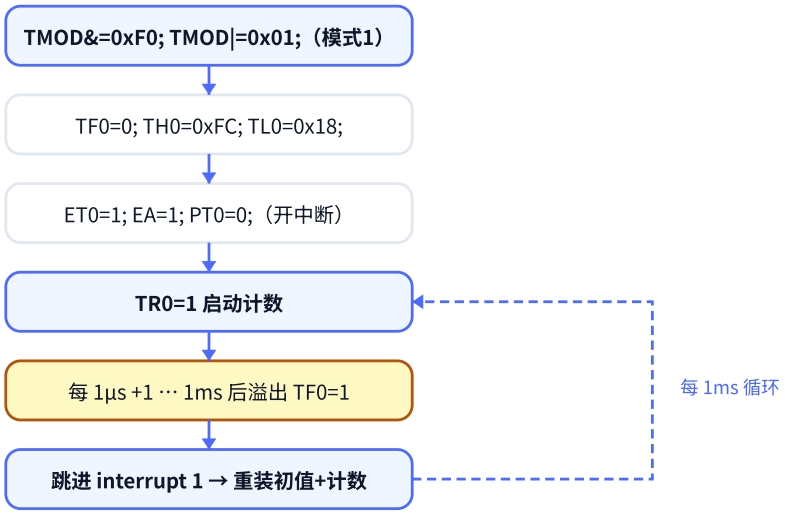


实物图 · 板上 12MHz 晶振：定时器的时钟来源，经 12T 分频后每 1μs 计数 +1 · 图源：网络搜索

易错点

- **忘了手动重装初值：**模式1 非自动重装，每次进中断都要 `TH0=0xFC; TL0=0x18;`，否则第二次起定时变成 65ms（P18 17:30、32:51）。
- **TMOD 直接整体赋值：** `TMOD=0x01` 会把高 4 位(T1)清掉；T0/T1 并存要用 `TMOD&=0xF0; TMOD|=0x01;` 与或式（P18 19:12）。

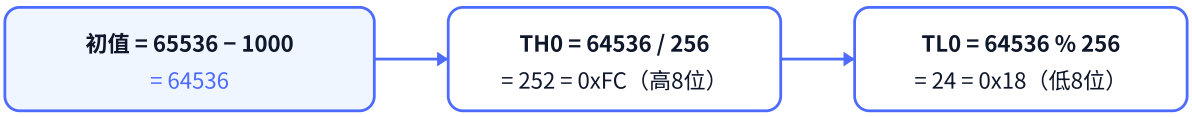
- **只开 ET0 不开 EA：**EA 是总中断开关，EA=0 时所有中断都进不来；ET0=1 与 EA=1 缺一不可（P17 58:16）。
- **初值手算口径：**定 N μs 装 65536−N（不是 65535−N）；手算 0xFC17 比计算器 0xFC18 差 1μs，以后用 STC-ISP 计算器生成（P18 28:30）。
- **计算器代码的 AUXR 句：**89C51 没有 AUXR 寄存器，生成代码第一句要删掉，再手动补 ET0/EA（P18 27:07、32:15）。
- **中断函数写长任务：**ISR 要短，LCD_ShowNum 刷屏这种慢操作放 while(1) 主循环，别进中断（P18 58:23）。
- **引脚别记混：**按键是 P3（K1=P3.1），P2 是 LED；老师口误过 P2 要改回 P3（P18 39:26–45:04）。
- **模块别一起调：**按键/定时器模块写完立即单独测试，别和定时器一起调，否则出问题分不清哪边（P18 45:04）。
- **换芯片查手册：**不同型号定时器个数/中断资源不同，别默认有 T1（P17 10:04）。



配置一次(Timer0_Init)，之后硬件每 1ms 自动敲门；ISR 里先重装 TH0/TL0 再数满 1000 次=1s（P18 02:39–17:30）

图 ②　定时器初始化+运行流程：配 TMOD→装初值→开 ET0/EA→TR0=1 启动，1ms 后溢出跳 interrupt 1，进去先手动重装 TH0/TL0 再做计数（P18 02:39–17:30、32:51）。

1ms 定时初值怎么算（12MHz / 12T / 模式1）

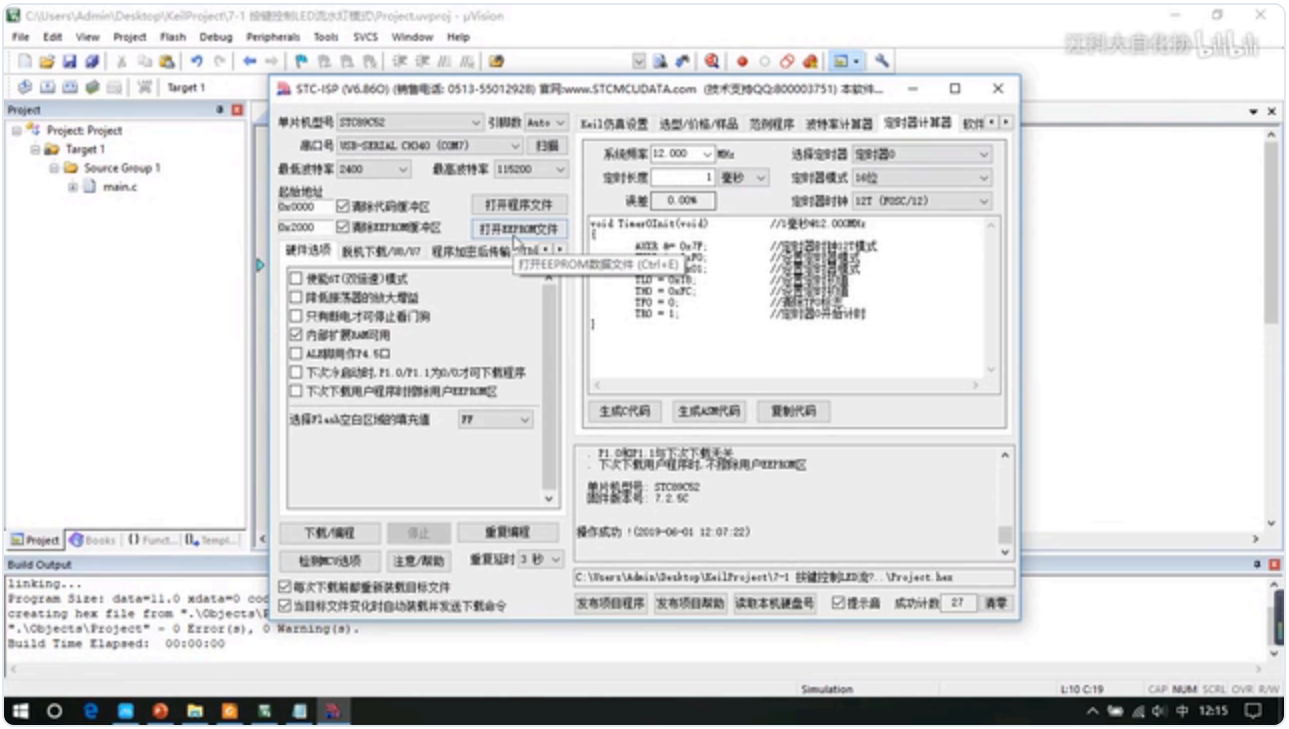


合成 TH0:TL0 = 0xFC18

口径：装 65536−N 而非 65535−N；手算 64535→0xFC17 比计算器差 1μs，以后直接用 STC-ISP 计算器

定 N μs 的通式：初值 = 65536 − N；一次最多 65536μs≈65.5ms（P18 08:51、09:33、28:30）

图 ③　16 位初值计算：65536−1000=64536，按 256 拆高 8 位 TH0=0xFC、低 8 位 TL0=0x18，合成 0xFC18；手算 0xFC17 与计算器差 1μs（P18 08:51–09:33、28:30–30:53）。



课程截图 · 仅供个人学习 · P18 26:42 —— STC-ISP 定时器计算器：系统频率12.000MHz、定时器0、定时1ms、16位、12T，生成代码 TL0=0x18/TH0=0xFC（即 0xFC18），首句 AUXR 在 89C52 要删

常见题型

题型	做法要点
算定时初值	12MHz/12T → 1μs/脉冲；定 N μs：初值=65536−N，TH0=初值/256、TL0=初值%256；1ms→0xFC18
写 Timer0_Init	TMOD&=0xF0;|=0x01 → TF0=0 → TH0/TL0 装初值 → TR0=1 → ET0=1 → EA=1 → PT0=0
写中断服务函数	`void 名() interrupt 1`；进去先手动重装 TH0/TL0，再 static 计数变量数满阈值做动作
定 1 秒/500ms	每 1ms 中断，ISR 里 static 计数；≥1000 做秒动作、≥500 做半秒动作，到值清零
T0/T1 并存配置	不要 TMOD=0x01，用 TMOD&=0xF0;|=0x01 只改低 4 位、保留 T1 高 4 位
按键切流水灯模式	main 扫 Key()，K1 切 LED_Mode 0/1；ISR 每 500ms 用 _crol_/_cror_ 循环移位
定时器时钟	ISR 只做 Sec/Min/Hour 进位(60/60/24)；LCD_ShowNum 刷屏放 while(1) 主循环

例题速览

下面 C 代码均为**自写示例（非原课程源码）**，按标准 Keil C51 / 8051 语法组织；初值 0xFC18 可由 STC-ISP 定时器计算器(12MHz/1ms/Timer0/16位/12T)生成，讲义只写骨架、不硬抄计算器内部配置句。

例 1（1ms 中断 + 1 秒翻转 D1） 用定时器每 1ms 进一次中断，数满 1000 次即 1 秒，翻转 D1。

- **思路：**Timer0_Init 配模式1、装 0xFC18 初值、开 ET0/EA、TR0=1；interrupt 1 里先重装初值，再 static 计数满 1000 次翻转 P2_0。
- **做法：**

```
#include <regx52.h>   /* 自写示例（非原课程源码） */

void Timer0_Init(void)
{
    TMOD &= 0xF0;   /* 清低4位(T0)，保留高4位(T1) */
    TMOD |= 0x01;   /* GATE=0 C/T=0 M1M0=01 → 模式1 16位定时 */
    TF0 = 0;        /* 清溢出标志，防误触发 */
    TH0 = 0xFC;     /* 1ms 初值高8位：65536-1000=64536=0xFC18 */
    TL0 = 0x18;     /* 低8位 */
    TR0 = 1;        /* 启动 T0 */
    ET0 = 1;        /* 允许 T0 中断 */
    EA = 1;         /* 开总中断 */
    PT0 = 0;        /* 低优先级 */
}

void Timer0_Routine() interrupt 1   /* T0 中断号 = 1 */
{
    static unsigned int T0_count;  /* 静态局部：函数退出后值保留 */
    TH0 = 0xFC;         /* 模式1非自动重装，每次进中断先重装 */
    TL0 = 0x18;
    T0_count++;
    if(T0_count >= 1000)   /* 1ms × 1000 = 1s */
    {
        T0_count = 0;
        P2_0 = ~P2_0;   /* 每秒翻转 D1 */
    }
}

void main()
{
    Timer0_Init();
    while(1)
    {
    }
}
```

- **结果：**D1 每 1 秒亮灭翻转一次，CPU 在 while(1) 里空着、不浪费；把 P2_0=0 单独写进 ISR 可验证程序真的跳进了中断（主函数没调用它）。

例 2（按键切换流水灯左右移模式） 定时器做 500ms 节拍，K1 在主循环切换左移/右移。

- **思路：**流水灯移位放 ISR（每 500ms 一次），按键扫描放 while(1)——两者并行，CPU 不再被 delay 卡死；K1 按一下 LED_Mode 在 0/1 间切。
- **做法：**

```
#include <regx52.h>   /* 自写示例（非原课程源码） */
#include <intrins.h> /* _crol_/_cror_ 循环移位 */
#include "Key.h"     /* 第3章独立按键模块（沿用） */

unsigned char KeyNum;
unsigned char LED_Mode;

void main()
{
    P2 = 0xFE;       /* D1 初始亮 */
    Timer0_Init();  /* 例1的初始化（内部每 500ms 动一次） */
    while(1)
    {
        KeyNum = Key();          /* 主循环扫按键 */
        if(KeyNum == 1)          /* K1=P3.1 */
        {
            LED_Mode++;
            if(LED_Mode >= 2) LED_Mode = 0;
        }
    }
}

void Timer0_Routine() interrupt 1
{
    static unsigned int T0_count;
    TH0 = 0xFC; TL0 = 0x18;     /* 每次进中断先重装 */
    T0_count++;
    if(T0_count >= 500)          /* 每 500ms 走一位 */
    {
        T0_count = 0;
        if(LED_Mode == 0)      P2 = _crol_(P2, 1);   /* 循环左移 */
        else if(LED_Mode == 1) P2 = _cror_(P2, 1);  /* 循环右移 */
    }
}
```

- **结果：**流水灯在 D1–D8 间循环跑动，按 K1 切换左/右方向；因节拍由定时器产生，主循环随时能响应按键，不再像 Delay(500) 那样"转起来就扫不到键"。

例 3（定时器时钟 · LCD1602 显示） 中断里做秒/分/时进位，LCD 刷屏放主循环。

- **思路：**ISR 只做最短的进位计数（快），把慢的 LCD_ShowNum 刷屏放在 while(1)，避免长任务堵中断。
- **做法：**

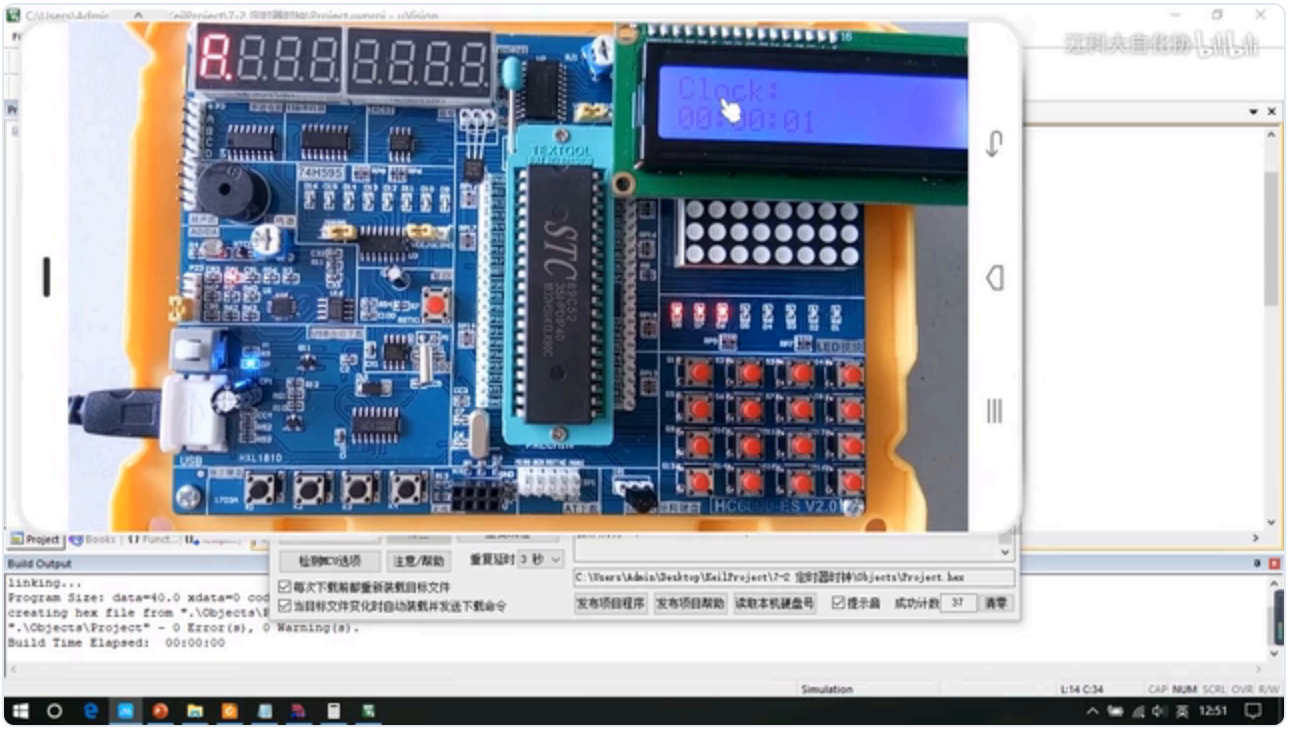
```
#include <regx52.h>   /* 自写示例（非原课程源码） */
#include "LCD1602.h" /* 第5章 LCD 模块（沿用） */

unsigned char Sec, Min, Hour;   /* 全局，上电默认 0 */

void main()
{
    LCD_Init();
    LCD_ShowString(1, 1, "Clock");
    Timer0_Init();
    while(1)   /* 慢操作(刷屏)放主循环 */
    {
        LCD_ShowNum(1, 1, Hour, 2);  LCD_ShowString(1, 3, ":");
        LCD_ShowNum(1, 4, Min, 2);   LCD_ShowString(1, 6, ":");
        LCD_ShowNum(1, 7, Sec, 2);
    }
}

void Timer0_Routine() interrupt 1
{
    static unsigned int T0_count;
    TH0 = 0xFC; TL0 = 0x18;
    T0_count++;
    if(T0_count >= 1000)   /* 每秒进一次 */
    {
        T0_count = 0;
        Sec++;
        if(Sec >= 60){ Sec=0; Min++; }
        if(Min >= 60){ Min=0; Hour++; }
        if(Hour >= 24){ Hour=0; }
    }
}
```

- **结果：**LCD 第一行显示 HH:MM:SS 并正常走时；可手动赋 Hour=23、Min=59、Sec=55 验证 59→00 的进位与 24 点归零。



课程截图 · 仅供个人学习 · P18 01:02:17 —— LCD 实屏"Clock"走时显示时:分:秒（秒位=01，每隔60秒向分进位）

结果速记：Timer0_Init = `TMOD&=0xF0;/|=0x01 → TF0=0 → TH0=0xFC/TL0=0x18 → TR0=1 → ET0=1 → EA=1 → PT0=0` ；ISR = `void Timer0_Routine() interrupt 1` ，进去**先重装 TH0/TL0**、再 static 计数。1ms 初值 **0xFC18**（65536−1000）；1ms×1000=1s、×500=0.5s。流水灯用 `_crol_/_cror_` 、时钟 ISR 只进位把刷屏留主循环。

一分钟自测

1. 12MHz 晶振、12T 分频下，定时器计数脉冲周期是多少？模式1 一次溢出最长多少毫秒？
2. 定 1ms 的初值是多少？拆成 TH0、TL0 各是多少（十六进制）？
3. 为什么模式1 每次进中断都要手动重装 TH0/TL0？不重装会怎样？
4. Timer0_Init 里 ET0=1 和 EA=1 分别起什么作用？只写 ET0=1 行不行？
5. T0/T1 同时用时，为什么不能直接写 TMOD=0x01？应该怎么写？
6. 定时器时钟例里，为什么把 LCD_ShowNum 刷屏放 while(1) 而不放中断服务函数？

答：

1. 12MHz÷12 = 1MHz，计数脉冲周期 = **1μs**；模式1 一次溢出最长 **65536μs≈65.5ms**。
2. 初值 = 65536−1000 = 64536 = **0xFC18**； `TH0=0xFC` （高8位）、 `TL0=0x18` （低8位）。
3. 模式1 **不是自动重装**，溢出后从 0 开始数；不重装，第二次起要数满 65536 个脉冲才溢出，定时就变成 65ms、全错。

4. `ET0=1` 是允许 T0 这一路中断， `EA=1` 是开总中断开关；**只写 ET0=1 不行**，EA=0 时所有中断都进不来。

5. `TMOD=0x01` 会把高 4 位(T1 配置)清掉；应写成 `TMOD&=0xF0; TMOD|=0x01;` ，只改低 4 位、保留 T1。

6. 中断服务函数要短，LCD_ShowNum 刷屏慢、耗时长，放 ISR 会拖慢甚至错过下一次 1ms 节拍；把快的进位计数放 ISR、慢的显示留主循环。

本节小结

两章串成一条"**从空转延时到硬件定时**"的升级线：**时钟链**（12MHz→12T→1μs 脉冲）→ **计数链**（TH0:TL0 16 位从初值 0xFC18 加到溢出）→ **中断链**（TF0=1 → interrupt 1 → 手动重装 + 软件数满 1000=1s）。综合例把它用活：流水灯节拍交给定时器、按键扫描留主循环，二者并行；时钟只在中断里做最简短位。记住三个反复出现的坑：**初值用 65536−N 且优先用计算器、每次进中断必须重装 TH0/TL0、TMOD 用与或式且 ET0/EA 都要开**。下一章进入下一个外设主题。